Assignment 2 — Planning & Architecture

IoT Telemetry Pipeline & Dashboard

System Architecture

The system is split across four logical tiers that communicate over three distinct links, each carrying a different protocol and serialization format. The diagram below maps every component and labels each link explicitly.

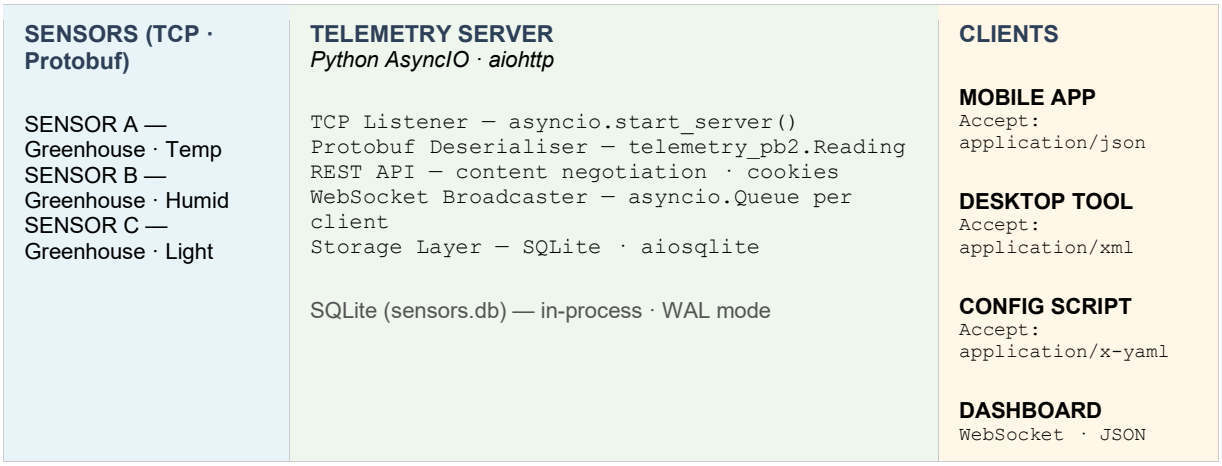

Figure 1 — System architecture with protocols and serialization formats on each link

LEGEND

- TCP · Protobuf (sensor ingress)
- HTTP/REST (historical queries)
- WebSocket (live feed)

The sensor simulators connect to the server over TCP and push Protobuf-encoded Reading messages at their configured interval. Protobuf was chosen here because the binary encoding is compact and cheap to parse on constrained hardware — exactly what low-cost greenhouse controllers need.

The REST API deliberately supports three formats (JSON, XML, YAML) through content negotiation rather than separate endpoints, so the mobile app, the legacy desktop tool and any config scripts can all be served by the same handler. The WebSocket feed is JSON-only because browsers have native JSON support and adding format negotiation to a live-push channel would add complexity with no real benefit.

Sequence Diagram — Sensor to Live Dashboard

The diagram below traces the full lifecycle of a single sensor reading: from the initial WebSocket upgrade handshake, through Protobuf ingestion and database storage, to the live dashboard rendering the value.

SENSOR	SERVER	STORAGE	DASHBOARD
	WebSocket Upgrade Handshake		
	HTTP GET /live Upgrade: websocket →		
	← 101 Switching Protocols · WS open		
	Sensor Ingestion		
TCP connect() →			
Protobuf Reading frame (sensor_id · timestamp · value · unit) →			
	→ INSERT INTO readings		
		← lastrowid · OK	
			← WS push · JSON {sensor_id, value, timestamp} render on chart
	Historical REST Query		
	GET /sensors/{id}/readings?from=…&to=… Accept: application/json · Cookie: session_id=… →		
	validate cookie → SELECT readings WHERE …		
		← ResultSet rows	
	← 200 OK · JSON readings array Set-Cookie: session_id=… (first request only)		
Sensor repeats Protobuf frame every report_interval_s seconds →			

Figure 2 — Full message lifecycle from sensor to live dashboard, including WebSocket upgrade

A few points worth noting from the sequence above. The WebSocket handshake happens once, up front, before any sensor data arrives — the dashboard stays connected and just waits for the server to push new readings. On the ingestion side, the server always writes to storage before broadcasting to WebSocket clients, so there is never a situation where a reading appears on the live feed but is missing from the historical query.

Data Model

Protobuf Schema — proto/telemetry.proto

Two messages are defined. Reading is the high-frequency message sent every few seconds over TCP. SensorInfo is sent once when the simulator first connects and mirrors what is registered via POST /sensors.

```
syntax = "proto3";
package telemetry;

// Single sensor reading — transmitted every report_interval_s seconds
message Reading {
  string sensor_id = 1;   // matches Sensor.id
  int64  timestamp  = 2;  // Unix epoch milliseconds (UTC)
  double value      = 3;  // numeric measurement
  string unit       = 4;  // "celsius", "percent", "lux", etc.
}

// Sent once at TCP connect; mirrors the REST POST /sensors body
message SensorInfo {
  string id          = 1;
  string location    = 2;  // e.g. "Greenhouse A"
  string sensor_type = 3;  // temperature | humidity | light | soil
  int32  interval_s  = 4;  // report interval from sensors.yaml
}
```

SQLite Schema

Two tables are sufficient. The sensors table acts as the registry; readings stores the time-series data with a foreign key back to the sensor. A composite index on (sensor_id, timestamp) makes time-range queries fast without needing a full table scan.

```
-- sensors: registered devices
CREATE TABLE sensors (
  id            TEXT    PRIMARY KEY,
  location      TEXT    NOT NULL,
  sensor_type   TEXT    NOT NULL,
  interval_s    INTEGER NOT NULL DEFAULT 10,
  registered_at INTEGER NOT NULL  -- Unix ms
);

-- readings: time-series measurements
CREATE TABLE readings (
  id        INTEGER PRIMARY KEY AUTOINCREMENT,
  sensor_id TEXT    NOT NULL REFERENCES sensors(id),
  timestamp INTEGER NOT NULL,  -- Unix ms
  value     REAL    NOT NULL,
  unit      TEXT    NOT NULL
);

-- index for fast time-range queries per sensor
CREATE INDEX idx_readings_sensor_time
  ON readings (sensor_id, timestamp);
```

REST Representations

The same underlying data is serialised differently depending on the client's Accept header. Below are the canonical JSON shapes; the XML and YAML equivalents carry the same fields.

SENSOR OBJECT

```
{
  "id":           "gh_a_temp_01",
  "location":     "Greenhouse A",
  "sensor_type":  "temperature",
  "interval_s":   10,
  "registered_at": 1715200000000
}
```

READING OBJECT

```
{
  "sensor_id": "gh_a_temp_01",
  "timestamp": 1715200060000,
  "value":     24.7,
  "unit":      "celsius"
}
```

REST Endpoint Specification

All six endpoints sit on the same aiohttp application instance. Every response honours the Accept header (JSON by default). A session_id cookie is issued on first contact and verified on all subsequent calls — the cookie value itself carries no state; it is purely for client identification in logs.

Path	Method	Request Body / Params	Response Body	Status Codes
/sensors	GET	—	Array of Sensor objects	200, 204 (none registered)
/sensors/{id}	GET	—	Single Sensor object	200, 404
/sensors	POST	JSON body: {id, location, sensor_type, interval_s}	Created Sensor object	201, 400 (malformed), 409 (duplicate id)
/sensors/{id}	DELETE	—	Empty body	204, 404
/sensors/{id}/readings	GET	Query: from, to (Unix ms, optional)	Array of Reading objects	200, 400 (bad params), 404
/live	GET (WS upgrade)	WebSocket upgrade headers	101 then JSON stream	101, 400

Content Negotiation

The Accept header is parsed using aiohttp's quality-value handling. Supported types are `application/json`, `application/xml`, and `application/x-yaml`. If none of the client's preferences match, the server falls back to JSON and still returns 200 — no 406 is issued, because strict rejection would break older clients that send wildcard Accept headers without listing specific types.

Example: Querying Historical Readings as XML

```
GET /sensors/gh_a_temp_01/readings?from=1715100000000&to=1715200000000 HTTP/1.1
Host: localhost:8080
Accept: application/xml
Cookie: session_id=a3f9c12d

HTTP/1.1 200 OK
Content-Type: application/xml

<readings>
  <reading>
    <sensor_id>gh_a_temp_01</sensor_id>
    <timestamp>1715100010000</timestamp>
    <value>23.4</value>
    <unit>celsius</unit>
  </reading>
  ...
</readings>
```

YAML Configuration Schema

The sensor simulator reads config/sensors.yaml at startup. YAML suits this role well — it is easy to edit by hand, supports inline comments, and parses directly into Python dicts and lists without a compilation step. The file is small and only read once, so YAML's verbosity relative to binary formats is not a concern here.

Worked Example — config/sensors.yaml

```
server:
  host: "127.0.0.1"  # TCP address of the telemetry server
  port: 9000

sensors:
  - id: "gh_a_temp_01"       # unique identifier (used in all messages and URLs)
    location: "Greenhouse A"  # human-readable label
    sensor_type: "temperature"  # one of: temperature | humidity | light | soil
    unit: "celsius"           # echoed in every Reading proto message
    report_interval_s: 5      # seconds between Protobuf pushes (integer, minimum 1)
    min_value: 15.0           # lower bound for simulated readings
    max_value: 40.0

  - id: "gh_b_hum_01"
    location: "Greenhouse B"
    sensor_type: "humidity"
    unit: "percent"
    report_interval_s: 10
    min_value: 40.0
```

```
    max_value: 95.0

  - id: "gh_c_light_01"
    location: "Greenhouse C"
    sensor_type: "light"
    unit: "lux"
    report_interval_s: 15
    min_value: 0.0
    max_value: 100000.0
```

Field	Type	Required	Description
server.host	string	Yes	Hostname or IP of the telemetry server
server.port	int 1–65535	Yes	TCP port the server is listening on
sensors[].id	string	Yes	Unique sensor identifier; used in Protobuf messages and REST URLs
sensors[].location	string	Yes	Human label for the physical location
sensors[].sensor_type	enum	Yes	temperature | humidity | light | soil
sensors[].unit	string	Yes	Unit string embedded in every Reading message
sensors[].report_interval_s	int ≥1	Yes	Time between consecutive Protobuf frames in seconds
sensors[].min_value	float	Yes	Lower bound for the random value generator
sensors[].max_value	float	Yes	Upper bound for the random value generator

Concurrency Model

The entire server runs on a single OS thread using Python's AsyncIO event loop. This is the right choice here because the workload is almost entirely I/O-bound — the server spends most of its time waiting for socket reads, database writes, or WebSocket sends rather than doing CPU-heavy work. AsyncIO handles this by running many coroutines concurrently on one thread: whenever a coroutine hits an await, the event loop hands control to whichever coroutine is next ready to run. There are no locks and no shared-state races because no two coroutines ever run at exactly the same moment.

Coroutines and Their Roles

HANDLE_SENSOR_CONNECTION(READER, WRITER)

- One coroutine spawned per incoming TCP connection by asyncio.start_server()
- Reads length-prefixed Protobuf frames in a loop, deserialises each to a Reading object
- Calls await db.store_reading(r) then broadcaster.publish(r)
- Catches ConnectionResetError to clean up when a sensor disconnects

WEBSOCKET_HANDLER(REQUEST)

- Upgrades the HTTP connection to a WebSocket via aiohttp
- Registers a per-client asyncio.Queue(maxsize=100) with the broadcaster
- Runs a writer coroutine: await queue.get() then ws.send_json()
- Deregisters the queue when the client disconnects or an error is raised

REST_HANDLER(REQUEST)

- Single async function registered to all REST routes
- Parses Accept header, queries await db.get_readings(…)
- Serialises the result to JSON, XML, or YAML
- Sets or validates the session_id cookie before returning

BROADCASTER.PUBLISH(READING)

- Called synchronously from within handle_sensor_connection — no await
- Iterates all registered client queues and calls put_nowait(reading)
- If a queue is full, the reading is dropped for that client — see slow consumer note below
- Never blocks, so the sensor ingestion loop is never slowed by a slow dashboard client

Slow consumer policy: Each WebSocket client has a bounded queue of 100 items. If a client's network is slow or the browser tab is in the background and can't drain the queue fast enough, put_nowait() raises QueueFull and that reading is silently dropped for that client only. The sensor ingestion path and all other connected clients are completely unaffected. The client will still receive the next reading once it catches up. This is a deliberate trade-off: live data is only valuable when it is actually live, so dropping stale frames is preferable to blocking the whole pipeline.

Task Structure at Runtime

```
asyncio.run(main())
|
+-- asyncio.start_server()  # one coroutine per TCP sensor connection
|   +-- handle_sensor_connection()
|       +-- await reader.read()         # yields while waiting for bytes
|       +-- await db.store()            # yields while SQLite write completes
|       +-- broadcaster.publish()       # synchronous: put_nowait(), never blocks
|
+-- aiohttp.AppRunner (REST + WebSocket)
    +-- rest_handler()
    |   +-- await db.get_readings()     # yields during SELECT
    +-- websocket_handler()
        +-- await queue.get()           # yields until broadcaster pushes a reading
```

Failure Handling

The guiding principle is that failures should be isolated to the component that caused them. A crashing sensor coroutine should not take down the REST API, and a slow WebSocket client should not block sensor ingestion. The sections below describe how each failure mode is handled.

Sensor Disconnects

When a sensor drops its TCP connection, the `await reader.read()` call inside `handle_sensor_connection()` raises either `ConnectionResetError` or `asyncio.IncompleteReadError`. Both are caught in a try/except block; the coroutine logs the disconnect and returns cleanly. The event loop removes it automatically and no resources are leaked. All historical readings from that sensor remain intact in SQLite. When the sensor reconnects, `asyncio.start_server()` spawns a fresh coroutine for it without any manual intervention.

Storage Layer Failures

All database calls are wrapped in `try/except aiosqlite.Error`. If a write fails during ingestion, the reading is dropped and the error is logged, but the sensor's TCP connection is kept alive — there is no reason to disconnect the sensor just because one write failed. If a SELECT fails during a REST query, the handler returns `503 Service Unavailable` with a JSON error body so the client receives a clear, machine-readable signal rather than a hang or a garbled response. SQLite is configured in WAL mode, which reduces contention between the concurrent read and write coroutines sharing the same in-process database.

Server Overload

WebSocket clients with full queues have their readings dropped silently as described in §F. The AsyncIO event loop naturally rate-limits concurrency since coroutines can only run one at a time — there is no thread-pool to exhaust. If the OS TCP backlog fills up because the server is genuinely overwhelmed, new sensor connection attempts will be queued by the kernel and the sensor simulator will retry. REST requests that arrive during a backlog are served as soon as a coroutine slot becomes available; if aiohttp's internal limits are hit, it returns `503`.

Malformed Protobuf Frames

Calling `ParseFromString()` on a corrupt buffer raises `google.protobuf.message.DecodeError`. This is caught per-frame inside `handle_sensor_connection()`; the frame is skipped and the error is logged with the sensor ID. A single corrupt frame does not disconnect the sensor — this could easily happen due to a transient network glitch and terminating on the first error would be overly aggressive. A counter tracks consecutive decode errors per connection; if it exceeds a configurable threshold (e.g. five in a row), the connection is closed gracefully on the assumption that something is fundamentally wrong with that sensor's encoding.

Error response format: All REST error responses use a consistent JSON envelope: `{"error": {"code": 503, "message": "storage unavailable", "detail": "…"}}`. This makes error handling straightforward for API clients regardless of whether they prefer JSON, XML, or YAML for successful responses.

Reference

- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, UC Irvine.
- Google. (2024). Protocol Buffers — Developer Guide. https://protobuf.dev/overview/
- Python Software Foundation. (2024). asyncio — Asynchronous I/O. https://docs.python.org/3/library/asyncio.html
- aiohttp contributors. (2024). aiohttp Documentation. https://docs.aiohttp.org/